

ПРИЛОЖЕНИЕ П

(обязательное)

Листинг проекта

В приложение вынесены исходные модули микро-ЭВМ и проверочные стенды. Код не включён в основной текст пояснительной записки, чтобы не нарушать структуру ПЗ.

П.1 Исходные модули проекта

alu_core.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity alu_core is
  port (
    a_i    : in word_t;
    b_i    : in word_t;
    flag_s_i : in std_logic;
    op_i    : in std_logic_vector(2 downto 0);
    y_o    : out word_t;
    z_o    : out std_logic;
    s_o    : out std_logic;
    c_o    : out std_logic;
    o_o    : out std_logic
  );
end entity;

architecture rtl of alu_core is
begin
  process(a_i, b_i, flag_s_i, op_i)
    variable a_u : unsigned(15 downto 0);
    variable sum17 : unsigned(16 downto 0);
    variable y_v : word_t;
    variable z_v : std_logic;
    variable s_v : std_logic;
    variable c_v : std_logic;
    variable o_v : std_logic;
  begin
    a_u := unsigned(a_i);
    y_v := a_i;
    c_v := '0';
    o_v := '0';
```

```

case op_i is
  when ALU_OR =>
    y_v := a_i or b_i;
  when ALU_NOR =>
    y_v := not (a_i or b_i);
  when ALU_SRA =>
    y_v := a_i(15) & a_i(15 downto 1);
    c_v := a_i(0);
  when ALU_INCS =>
    sum17 := ('0' & a_u);
    if flag_s_i = '1' then
      sum17 := ('0' & a_u) + to_unsigned(1, 17);
    end if;
    y_v := std_logic_vector(sum17(15 downto 0));
    c_v := sum17(16);
    o_v := (not a_i(15)) and flag_s_i and y_v(15);
  when others =>
    y_v := a_i;
end case;

if y_v = x"0000" then
  z_v := '1';
else
  z_v := '0';
end if;
s_v := y_v(15);

y_o <= y_v;
z_o <= z_v;
s_o <= s_v;
c_o <= c_v;
o_o <= o_v;
end process;
end architecture;
branch_predictor.vhd
library ieee;
use ieee.std_logic_1164.all;

```

```
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;
```

```
entity branch_predictor is
```

```
port (
    clk_i      : in std_logic;
    rst_i      : in std_logic;
    query_valid_i : in std_logic;
    pc_query_i  : in addr_t;
    hist_o      : out std_logic_vector(1 downto 0);
    pred_taken_o : out std_logic;
    pred_target_o : out addr_t;
    update_valid_i : in std_logic;
    pc_update_i  : in addr_t;
    hist_i      : in std_logic_vector(1 downto 0);
    actual_taken_i : in std_logic;
    actual_tgt_i  : in addr_t
);
```

```
end entity;
```

```
architecture rtl of branch_predictor is
```

```
type pht_t is array (0 to 15) of unsigned(1 downto 0);
type target_t is array (0 to 15) of addr_t;
type valid_t is array (0 to 15) of std_logic;
```

```
signal pht      : pht_t := (others => "01");
signal btb_tgt  : target_t := (others => (others => '0'));
signal btb_valid : valid_t := (others => '0');
signal ghr_r    : std_logic_vector(1 downto 0) := "00";
```

```
function idx_of(pc : addr_t; hist : std_logic_vector(1 downto 0)) return integer is
    variable idx_v : std_logic_vector(3 downto 0);
```

```
begin
```

```
    idx_v := std_logic_vector(pc(1 downto 0)) & hist;
    return to_integer(unsigned(idx_v));
```

```
end function;
```

```
function sat_dec(v : unsigned(1 downto 0)) return unsigned is
```

```

begin
  if v = "00" then
    return v;
  else
    return v - 1;
  end if;
end function;
begin
  hist_o <= ghr_r;

  process(query_valid_i, pc_query_i, ghr_r, pht, btb_valid, btb_tgt)
    variable idx : integer;
  begin
    pred_taken_o <= '0';
    pred_target_o <= (others => '0');
    if query_valid_i = '1' then
      idx := idx_of(pc_query_i, ghr_r);
      if pht(idx)(1) = '1' and btb_valid(idx) = '1' then
        pred_taken_o <= '1';
        pred_target_o <= btb_tgt(idx);
      end if;
    end if;
  end process;

  process(clk_i)
    variable idx : integer;
  begin
    if rising_edge(clk_i) then
      if rst_i = '1' then
        pht      <= (others => "01");
        btb_tgt  <= (others => (others => '0'));
        btb_valid <= (others => '0');
        ghr_r    <= "00";
      elsif update_valid_i = '1' then
        idx := idx_of(pc_update_i, hist_i);
        if actual_taken_i = '1' then
          pht(idx)      <= sat_inc2(pht(idx));

```

```

        btb_tgt(idx) <= actual_tgt_i;
        btb_valid(idx) <= '1';
    else
        pht(idx) <= sat_dec(pht(idx));
    end if;
    ghr_r <= ghr_r(0) & actual_taken_i;
end if;
end process;
end architecture;
bus_arbiter_2master.vhd
library ieee;
use ieee.std_logic_1164.all;

entity bus_arbiter_2master is
    port (
        req_cpu_i  : in std_logic;
        req_dma_i  : in std_logic;
        grant_cpu_o : out std_logic;
        grant_dma_o : out std_logic
    );
end entity;

architecture rtl of bus_arbiter_2master is
begin
    grant_dma_o <= req_dma_i;
    grant_cpu_o <= req_cpu_i and (not req_dma_i);
end architecture;
cache_4way_age.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity cache_4way_age is
    port (
        clk_i      : in std_logic;

```

```

    rst_i      : in std_logic;

    cpu_req_i   : in std_logic;
    cpu_we_i    : in std_logic;
    cpu_addr_i  : in addr_t;
    cpu_wdata_i : in word_t;
    cpu_rdata_o : out word_t;
    cpu_ready_o : out std_logic;
    hit_o       : out std_logic;
    miss_o      : out std_logic;

    ram_req_o   : out std_logic;
    ram_we_o    : out std_logic;
    ram_addr_o  : out addr_t;
    ram_wdata_o : out word_t;
    ram_rdata_i : in word_t;
    ram_grant_i : in std_logic
);
end entity;

```

architecture rtl of cache_4way_age is

```

    subtype cache_tag_t is std_logic_vector(11 downto 0);
    type cache_state_t is (IDLE, WAIT_READ_GRANT, WAIT_READ_DATA,
WAIT_WRITE_GRANT);
    type set_way_word_t is array (0 to CACHE_SETS-1, 0 to CACHE_WAYS-1) of
word_t;
    type set_way_tag_t is array (0 to CACHE_SETS-1, 0 to CACHE_WAYS-1) of
cache_tag_t;
    type set_way_age_t is array (0 to CACHE_SETS-1, 0 to CACHE_WAYS-1) of
unsigned(1 downto 0);
    type set_way_valid_t is array (0 to CACHE_SETS-1, 0 to CACHE_WAYS-1) of
std_logic;

    signal state_r      : cache_state_t := IDLE;
    signal data_r       : set_way_word_t := (others => (others => '0'));
    signal tag_r        : set_way_tag_t := (others => (others => '0'));
    signal age_r        : set_way_age_t := (others => (others => '0'));
    signal valid_r      : set_way_valid_t := (others => (others => '0'));

```

```

signal req_addr_r : addr_t := (others => '0');
signal req_wdata_r : word_t := (others => '0');
signal req_we_r : std_logic := '0';
signal write_hit_r : std_logic := '0';
signal victim_r : integer range 0 to CACHE_WAYS-1 := 0;
signal set_r : integer range 0 to CACHE_SETS-1 := 0;
signal tag_req_r : cache_tag_t := (others => '0');

signal cpu_rdata_r : word_t := (others => '0');
signal cpu_ready_r : std_logic := '0';
signal hit_r : std_logic := '0';
signal miss_r : std_logic := '0';
signal ram_req_r : std_logic := '0';
signal ram_we_r : std_logic := '0';
signal ram_addr_r : addr_t := (others => '0');
signal ram_wdata_r : word_t := (others => '0');

function cache_set(addr : addr_t) return integer is
begin
    return to_integer(unsigned(addr(3 downto 0)));
end function;

function cache_tag(addr : addr_t) return cache_tag_t is
begin
    return addr(15 downto 4);
end function;

function inc_age(v : unsigned(1 downto 0)) return unsigned is
begin
    if v = "11" then
        return v;
    else
        return v + 1;
    end if;
end function;

begin
    cpu_rdata_o <= cpu_rdata_r;

```



```

cpu_ready_o <= cpu_ready_r;
hit_o      <= hit_r;
miss_o     <= miss_r;
ram_req_o  <= ram_req_r;
ram_we_o   <= ram_we_r;
ram_addr_o <= ram_addr_r;
ram_wdata_o <= ram_wdata_r;

process(clk_i)
    variable set_v   : integer range 0 to CACHE_SETS-1;
    variable hit_v   : boolean;
    variable hit_way : integer range 0 to CACHE_WAYS-1;
    variable victim_v : integer range 0 to CACHE_WAYS-1;
    variable tag_v   : std_logic_vector(11 downto 0);
begin
    if rising_edge(clk_i) then
        if rst_i = '1' then
            state_r   <= IDLE;
            data_r     <= (others => (others => (others => '0')));
            tag_r      <= (others => (others => (others => '0')));
            age_r      <= (others => (others => (others => '0')));
            valid_r    <= (others => (others => '0'));
            req_addr_r <= (others => '0');
            req_wdata_r <= (others => '0');
            req_we_r   <= '0';
            write_hit_r <= '0';
            victim_r   <= 0;
            set_r      <= 0;
            tag_req_r  <= (others => '0');
            cpu_rdata_r <= (others => '0');
            cpu_ready_r <= '0';
            hit_r      <= '0';
            miss_r     <= '0';
            ram_req_r  <= '0';
            ram_we_r   <= '0';
            ram_addr_r <= (others => '0');
            ram_wdata_r <= (others => '0');

```

```

else
    cpu_ready_r <= '0';
    hit_r      <= '0';
    miss_r     <= '0';

    case state_r is
        when IDLE =>
            ram_req_r <= '0';
            ram_we_r  <= '0';

            if cpu_req_i = '1' and cpu_ready_r = '0' then
                set_v := cache_set(cpu_addr_i);
                tag_v := cache_tag(cpu_addr_i);
                hit_v := false;
                hit_way := 0;
                victim_v := 0;

                for way in 0 to CACHE_WAYS-1 loop
                    if valid_r(set_v, way) = '1' and tag_r(set_v, way) = tag_v then
                        hit_v := true;
                        hit_way := way;
                    end if;
                end loop;

                for way in 0 to CACHE_WAYS-1 loop
                    if valid_r(set_v, way) = '0' then
                        victim_v := way;
                    elsif valid_r(set_v, victim_v) = '1' and age_r(set_v, way) > age_r(set_v,
victim_v) then
                        victim_v := way;
                    end if;
                end loop;

                req_addr_r <= cpu_addr_i;
                req_wdata_r <= cpu_wdata_i;
                req_we_r   <= cpu_we_i;
                write_hit_r <= '0';
                victim_r   <= victim_v;
            end if;
        end case;
    end if;
end process;

```

```

set_r      <= set_v;
tag_req_r  <= tag_v;

if hit_v and cpu_we_i = '0' then
    cpu_rdata_r <= data_r(set_v, hit_way);
    cpu_ready_r <= '1';
    hit_r      <= '1';
elsif cpu_we_i = '1' then
    if hit_v then
        data_r(set_v, hit_way) <= cpu_wdata_i;
        victim_v := hit_way;
        write_hit_r <= '1';
    end if;
    ram_req_r  <= '1';
    ram_we_r   <= '1';
    ram_addr_r <= cpu_addr_i;
    ram_wdata_r <= cpu_wdata_i;
    victim_r   <= victim_v;
    state_r    <= WAIT_WRITE_GRANT;
    if hit_v then
        hit_r <= '1';
    else
        miss_r <= '1';
    end if;
else
    ram_req_r  <= '1';
    ram_we_r   <= '0';
    ram_addr_r <= cpu_addr_i;
    ram_wdata_r <= (others => '0');
    state_r    <= WAIT_READ_GRANT;
    miss_r     <= '1';
end if;
end if;

when WAIT_READ_GRANT =>
    ram_req_r <= '1';
    ram_we_r  <= '0';

```

```

ram_addr_r <= req_addr_r;
if ram_grant_i = '1' then
    state_r <= WAIT_READ_DATA;
end if;

when WAIT_READ_DATA =>
    ram_req_r <= '0';
    ram_we_r <= '0';
    data_r(set_r, victim_r) <= ram_rdata_i;
    tag_r(set_r, victim_r) <= tag_req_r;
    valid_r(set_r, victim_r) <= '1';
    age_r(set_r, victim_r) <= (others => '0');
    for way in 0 to CACHE_WAYS-1 loop
        if way /= victim_r and valid_r(set_r, way) = '1' then
            age_r(set_r, way) <= inc_age(age_r(set_r, way));
        end if;
    end loop;
    cpu_rdata_r <= ram_rdata_i;
    cpu_ready_r <= '1';
    miss_r <= '1';
    state_r <= IDLE;

when WAIT_WRITE_GRANT =>
    ram_req_r <= '1';
    ram_we_r <= '1';
    ram_addr_r <= req_addr_r;
    ram_wdata_r <= req_wdata_r;
    if ram_grant_i = '1' then
        if write_hit_r = '0' then
            data_r(set_r, victim_r) <= req_wdata_r;
            tag_r(set_r, victim_r) <= tag_req_r;
            valid_r(set_r, victim_r) <= '1';
            age_r(set_r, victim_r) <= (others => '0');
        end if;
        for way in 0 to CACHE_WAYS-1 loop
            if way /= victim_r and valid_r(set_r, way) = '1' then
                age_r(set_r, way) <= inc_age(age_r(set_r, way));
            end if;
        end loop;
    end if;
end when;

```

```

        end loop;
    end if;
    cpu_ready_r <= '1';
    state_r    <= IDLE;
    ram_req_r  <= '0';
    ram_we_r   <= '0';
    end if;
end case;
end if;
end if;
end process;
end architecture;

```

cache_4way_structural_doc.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

```

-- Documentation-oriented structural cache model.
 -- It is intentionally split into named blocks so Quartus can draw
 -- a readable one-page cache structure instead of a 75-page primitive netlist.

```

entity cache_data_way_doc is
    port (
        clk_i   : in  std_logic;
        set_i   : in  std_logic_vector(3 downto 0);
        we_i    : in  std_logic;
        data_i  : in  word_t;
        data_o  : out word_t
    );
end entity;

```

```

architecture rtl of cache_data_way_doc is
    type data_mem_t is array (0 to CACHE_SETS-1) of word_t;
    signal mem_r : data_mem_t := (others => (others => '0'));
begin
    process(clk_i)

```

```

begin
  if rising_edge(clk_i) then
    if we_i = '1' then
      mem_r(to_integer(unsigned(set_i))) <= data_i;
    end if;
  end if;
end process;

data_o <= mem_r(to_integer(unsigned(set_i)));
end architecture;

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity cache_tag_way_doc is
  port (
    clk_i  : in  std_logic;
    set_i  : in  std_logic_vector(3 downto 0);
    we_i   : in  std_logic;
    tag_i  : in  std_logic_vector(11 downto 0);
    tag_o   : out std_logic_vector(11 downto 0)
  );
end entity;

architecture rtl of cache_tag_way_doc is
  type tag_mem_t is array (0 to CACHE_SETS-1) of std_logic_vector(11 downto 0);
  signal mem_r : tag_mem_t := (others => (others => '0'));
begin
  process(clk_i)
  begin
    if rising_edge(clk_i) then
      if we_i = '1' then
        mem_r(to_integer(unsigned(set_i))) <= tag_i;
      end if;
    end if;
  end process;
end architecture;

```

```
    tag_o <= mem_r(to_integer(unsigned(set_i)));  
end architecture;
```

```
library ieee;  
use ieee.std_logic_1164.all;  
use ieee.numeric_std.all;  
use work.microcomputer_pkg.all;
```

```
entity cache_valid_way_doc is  
    port (  
        clk_i   : in std_logic;  
        rst_i   : in std_logic;  
        set_i   : in std_logic_vector(3 downto 0);  
        we_i    : in std_logic;  
        valid_i  : in std_logic;  
        valid_o  : out std_logic  
    );  
end entity;
```

```
architecture rtl of cache_valid_way_doc is  
    type valid_mem_t is array (0 to CACHE_SETS-1) of std_logic;  
    signal mem_r : valid_mem_t := (others => '0');  
begin  
    process(clk_i)  
    begin  
        if rising_edge(clk_i) then  
            if rst_i = '1' then  
                mem_r <= (others => '0');  
            elsif we_i = '1' then  
                mem_r(to_integer(unsigned(set_i))) <= valid_i;  
            end if;  
        end if;  
    end process;  
  
    valid_o <= mem_r(to_integer(unsigned(set_i)));  
end architecture;
```

```
library ieee;
```

```

use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity cache_age_way_doc is
  port (
    clk_i  : in std_logic;
    rst_i  : in std_logic;
    set_i  : in std_logic_vector(3 downto 0);
    we_i   : in std_logic;
    inc_i  : in std_logic;
    age_i  : in unsigned(1 downto 0);
    age_o  : out unsigned(1 downto 0)
  );
end entity;

architecture rtl of cache_age_way_doc is
  type age_mem_t is array (0 to CACHE_SETS-1) of unsigned(1 downto 0);
  signal mem_r : age_mem_t := (others => (others => '0'));
begin
  process(clk_i)
    variable idx_v : integer range 0 to CACHE_SETS-1;
  begin
    if rising_edge(clk_i) then
      idx_v := to_integer(unsigned(set_i));
      if rst_i = '1' then
        mem_r <= (others => (others => '0'));
      elsif we_i = '1' then
        mem_r(idx_v) <= age_i;
      elsif inc_i = '1' then
        mem_r(idx_v) <= sat_inc2(mem_r(idx_v));
      end if;
    end if;
  end process;

  age_o <= mem_r(to_integer(unsigned(set_i)));
end architecture;

```



```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity cache_tag_compare_doc is
  port (
    req_tag_i : in std_logic_vector(11 downto 0);
    way_tag_i : in std_logic_vector(11 downto 0);
    equal_o    : out std_logic
  );
end entity;

architecture rtl of cache_tag_compare_doc is
begin
  equal_o <= '1' when way_tag_i = req_tag_i else '0';
end architecture;

```

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity cache_hit_miss_logic_doc is
  port (
    tag_equal0_i : in std_logic;
    tag_equal1_i : in std_logic;
    tag_equal2_i : in std_logic;
    tag_equal3_i : in std_logic;
    valid0_i     : in std_logic;
    valid1_i     : in std_logic;
    valid2_i     : in std_logic;
    valid3_i     : in std_logic;
    way_hit_o    : out std_logic_vector(3 downto 0);
    hit_o        : out std_logic;
    miss_o       : out std_logic
  );
end entity;

```

```

architecture rtl of cache_hit_miss_logic_doc is
    signal way_hit_s : std_logic_vector(3 downto 0);
    signal hit_s      : std_logic;
begin
    -- These four ANDs plus OR/NOT are the useful "primitives" to show in the report.
    way_hit_s(0) <= tag_equal0_i and valid0_i;
    way_hit_s(1) <= tag_equal1_i and valid1_i;
    way_hit_s(2) <= tag_equal2_i and valid2_i;
    way_hit_s(3) <= tag_equal3_i and valid3_i;
    hit_s        <= way_hit_s(0) or way_hit_s(1) or way_hit_s(2) or way_hit_s(3);

    way_hit_o <= way_hit_s;
    hit_o      <= hit_s;
    miss_o     <= not hit_s;
end architecture;

```

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

```

```

entity cache_hit_mux_doc is
    port (
        hit0_i : in std_logic;
        hit1_i : in std_logic;
        hit2_i : in std_logic;
        hit3_i : in std_logic;
        data0_i : in word_t;
        data1_i : in word_t;
        data2_i : in word_t;
        data3_i : in word_t;
        hit_way_o: out std_logic_vector(1 downto 0);
        data_o   : out word_t
    );
end entity;

```

```

architecture rtl of cache_hit_mux_doc is

```

```

begin
process(hit0_i, hit1_i, hit2_i, hit3_i, data0_i, data1_i, data2_i, data3_i)
begin
    if hit0_i = '1' then
        data_o  <= data0_i;
        hit_way_o <= "00";
    elsif hit1_i = '1' then
        data_o  <= data1_i;
        hit_way_o <= "01";
    elsif hit2_i = '1' then
        data_o  <= data2_i;
        hit_way_o <= "10";
    else
        data_o  <= data3_i;
        hit_way_o <= "11";
    end if;
end process;
end architecture;

```

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

```

```

entity cache_address_split_doc is
port (
    cpu_addr_i : in  addr_t;
    tag_o      : out std_logic_vector(11 downto 0);
    set_o      : out std_logic_vector(3  downto 0)
);
end entity;

```

```

architecture rtl of cache_address_split_doc is
begin
    tag_o <= cpu_addr_i(15 downto 4);
    set_o <= cpu_addr_i(3  downto 0);
end architecture;

```

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity cache_victim_select_doc is
  port (
    valid0_i : in std_logic;
    valid1_i : in std_logic;
    valid2_i : in std_logic;
    valid3_i : in std_logic;
    age0_i   : in unsigned(1 downto 0);
    age1_i   : in unsigned(1 downto 0);
    age2_i   : in unsigned(1 downto 0);
    age3_i   : in unsigned(1 downto 0);
    victim_o  : out std_logic_vector(1 downto 0);
    onehot_o  : out std_logic_vector(3 downto 0)
  );
end entity;

architecture rtl of cache_victim_select_doc is
begin
  process(valid0_i, valid1_i, valid2_i, valid3_i, age0_i, age1_i, age2_i, age3_i)
  begin
    if valid0_i = '0' then
      victim_o <= "00";
      onehot_o <= "0001";
    elsif valid1_i = '0' then
      victim_o <= "01";
      onehot_o <= "0010";
    elsif valid2_i = '0' then
      victim_o <= "10";
      onehot_o <= "0100";
    elsif valid3_i = '0' then
      victim_o <= "11";
      onehot_o <= "1000";
    elsif age1_i >= age0_i and age1_i >= age2_i and age1_i >= age3_i then

```

```

    victim_o <= "01";
    onehot_o <= "0010";
    elsif age2_i >= age0_i and age2_i >= age1_i and age2_i >= age3_i then
        victim_o <= "10";
        onehot_o <= "0100";
    elsif age3_i >= age0_i and age3_i >= age1_i and age3_i >= age2_i then
        victim_o <= "11";
        onehot_o <= "1000";
    else
        victim_o <= "00";
        onehot_o <= "0001";
    end if;
end process;
end architecture;

```

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

```

```

entity cache_control_doc is
    port (
        clk_i      : in std_logic;
        rst_i      : in std_logic;
        cpu_req_i   : in std_logic;
        cpu_we_i    : in std_logic;
        hit_i       : in std_logic;
        hit_way_i   : in std_logic_vector(1 downto 0);
        victim_i    : in std_logic_vector(1 downto 0);
        victim_oh_i : in std_logic_vector(3 downto 0);
        ram_grant_i : in std_logic;
        way_we_o    : out std_logic_vector(3 downto 0);
        age_inc_o   : out std_logic_vector(3 downto 0);
        ram_req_o   : out std_logic;
        ram_we_o    : out std_logic;
        fill_sel_o  : out std_logic;
        cpu_ready_o : out std_logic
    );
end entity;

```

```
);  
end entity;
```

architecture rtl of cache_control_doc is

```
    type state_t is (IDLE, HIT_READ, MISS_READ_REQ, MISS_FILL,  
WRITE_THROUGH);
```

```
    signal state_r : state_t := IDLE;
```

```
begin
```

```
    process(clk_i)
```

```
    begin
```

```
        if rising_edge(clk_i) then
```

```
            if rst_i = '1' then
```

```
                state_r <= IDLE;
```

```
            else
```

```
                case state_r is
```

```
                    when IDLE =>
```

```
                        if cpu_req_i = '1' and cpu_we_i = '0' and hit_i = '1' then
```

```
                            state_r <= HIT_READ;
```

```
                        elsif cpu_req_i = '1' and cpu_we_i = '0' then
```

```
                            state_r <= MISS_READ_REQ;
```

```
                        elsif cpu_req_i = '1' and cpu_we_i = '1' then
```

```
                            state_r <= WRITE_THROUGH;
```

```
                        else
```

```
                            state_r <= IDLE;
```

```
                        end if;
```

```
                    when HIT_READ =>
```

```
                        state_r <= IDLE;
```

```
                    when MISS_READ_REQ =>
```

```
                        if ram_grant_i = '1' then
```

```
                            state_r <= MISS_FILL;
```

```
                        end if;
```

```
                    when MISS_FILL =>
```

```
                        state_r <= IDLE;
```

```
                    when WRITE_THROUGH =>
```

```
                        if ram_grant_i = '1' then
```

```
                            state_r <= IDLE;
```

```
                        end if;
```

```

        end case;
    end if;
end if;
end process;

ram_req_o <= '1' when state_r = MISS_READ_REQ or state_r =
WRITE_THROUGH else '0';
ram_we_o <= '1' when state_r = WRITE_THROUGH else '0';
fill_sel_o <= '1' when state_r = MISS_FILL else '0';
cpu_ready_o <= '1' when state_r = HIT_READ or state_r = MISS_FILL or (state_r =
WRITE_THROUGH and ram_grant_i = '1') else '0';

way_we_o <= victim_oh_i when state_r = MISS_FILL else
    "0001" when state_r = WRITE_THROUGH and hit_i = '1' and hit_way_i =
"00" else
    "0010" when state_r = WRITE_THROUGH and hit_i = '1' and hit_way_i =
"01" else
    "0100" when state_r = WRITE_THROUGH and hit_i = '1' and hit_way_i =
"10" else
    "1000" when state_r = WRITE_THROUGH and hit_i = '1' and hit_way_i =
"11" else
    victim_oh_i when state_r = WRITE_THROUGH else
    "0000";

age_inc_o <= not victim_oh_i when state_r = MISS_FILL else "0000";
end architecture;

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity cache_ram_interface_doc is
port (
    cpu_addr_i : in addr_t;
    cpu_wdata_i : in word_t;
    cpu_we_i : in std_logic;
    ram_req_i : in std_logic;

```

```

    ram_we_i    : in std_logic;
    ram_req_o   : out std_logic;
    ram_we_o    : out std_logic;
    ram_addr_o  : out addr_t;
    ram_wdata_o : out word_t
);
end entity;

```

architecture rtl of cache_ram_interface_doc is

begin

```

    ram_req_o <= ram_req_i;
    ram_we_o  <= ram_we_i and cpu_we_i;
    ram_addr_o <= cpu_addr_i;
    ram_wdata_o <= cpu_wdata_i;
end architecture;

```

library ieee;

use ieee.std_logic_1164.all;

use ieee.numeric_std.all;

use work.microcomputer_pkg.all;

entity cache_4way_structural_doc is

port (

```

    clk_i      : in std_logic;
    rst_i      : in std_logic;
    cpu_req_i   : in std_logic;
    cpu_we_i    : in std_logic;
    cpu_addr_i  : in addr_t;
    cpu_wdata_i : in word_t;
    cpu_rdata_o : out word_t;
    cpu_ready_o : out std_logic;
    hit_o       : out std_logic;
    miss_o      : out std_logic;
    ram_req_o   : out std_logic;
    ram_we_o    : out std_logic;
    ram_addr_o  : out addr_t;
    ram_wdata_o : out word_t;

```



```

    ram_rdata_i : in word_t;
    ram_grant_i : in std_logic
);
end entity;

```

architecture structural of cache_4way_structural_doc is

```

    signal set_s      : std_logic_vector(3 downto 0);
    signal tag_s      : std_logic_vector(11 downto 0);
    signal data_in_s  : word_t;

    signal data0_s    : word_t;
    signal data1_s    : word_t;
    signal data2_s    : word_t;
    signal data3_s    : word_t;
    signal tag0_s     : std_logic_vector(11 downto 0);
    signal tag1_s     : std_logic_vector(11 downto 0);
    signal tag2_s     : std_logic_vector(11 downto 0);
    signal tag3_s     : std_logic_vector(11 downto 0);
    signal valid0_s   : std_logic;
    signal valid1_s   : std_logic;
    signal valid2_s   : std_logic;
    signal valid3_s   : std_logic;
    signal age0_s     : unsigned(1 downto 0);
    signal age1_s     : unsigned(1 downto 0);
    signal age2_s     : unsigned(1 downto 0);
    signal age3_s     : unsigned(1 downto 0);

    signal tag_equal0_s: std_logic;
    signal tag_equal1_s: std_logic;
    signal tag_equal2_s: std_logic;
    signal tag_equal3_s: std_logic;
    signal way_hit_s   : std_logic_vector(3 downto 0);
    signal hit_any_s   : std_logic;
    signal miss_s      : std_logic;
    signal hit_way_s   : std_logic_vector(1 downto 0);
    signal victim_s    : std_logic_vector(1 downto 0);
    signal victim_oh_s : std_logic_vector(3 downto 0);

```

```

signal way_we_s  : std_logic_vector(3 downto 0);
signal age_inc_s  : std_logic_vector(3 downto 0);
signal fill_sel_s : std_logic;
signal ram_req_s  : std_logic;
signal ram_we_s   : std_logic;
signal cache_data_s: word_t;
begin
    data_in_s <= ram_rdata_i when fill_sel_s = '1' else cpu_wdata_i;
    hit_o     <= hit_any_s;
    miss_o    <= miss_s;
    cpu_rdata_o <= cache_data_s when hit_any_s = '1' else ram_rdata_i;

U_ADDRESS_SPLIT_TAG_SET : entity work.cache_address_split_doc
    port map (
        cpu_addr_i => cpu_addr_i,
        tag_o      => tag_s,
        set_o      => set_s
    );

U_CACHE_CONTROL : entity work.cache_control_doc
    port map (
        clk_i      => clk_i,
        rst_i      => rst_i,
        cpu_req_i  => cpu_req_i,
        cpu_we_i   => cpu_we_i,
        hit_i      => hit_any_s,
        hit_way_i  => hit_way_s,
        victim_i   => victim_s,
        victim_oh_i => victim_oh_s,
        ram_grant_i => ram_grant_i,
        way_we_o   => way_we_s,
        age_inc_o  => age_inc_s,
        ram_req_o  => ram_req_s,
        ram_we_o   => ram_we_s,
        fill_sel_o => fill_sel_s,
        cpu_ready_o => cpu_ready_o
    );

```

```

U_VICTIM_SELECT : entity work.cache_victim_select_doc
port map (
    valid0_i => valid0_s,
    valid1_i => valid1_s,
    valid2_i => valid2_s,
    valid3_i => valid3_s,
    age0_i  => age0_s,
    age1_i  => age1_s,
    age2_i  => age2_s,
    age3_i  => age3_s,
    victim_o => victim_s,
    onehot_o => victim_oh_s
);

```

```

U_RAM_INTERFACE : entity work.cache_ram_interface_doc
port map (
    cpu_addr_i => cpu_addr_i,
    cpu_wdata_i => cpu_wdata_i,
    cpu_we_i   => cpu_we_i,
    ram_req_i  => ram_req_s,
    ram_we_i   => ram_we_s,
    ram_req_o  => ram_req_o,
    ram_we_o   => ram_we_o,
    ram_addr_o => ram_addr_o,
    ram_wdata_o => ram_wdata_o
);

```

```

U_DATA_MUX : entity work.cache_hit_mux_doc
port map (
    hit0_i  => way_hit_s(0),
    hit1_i  => way_hit_s(1),
    hit2_i  => way_hit_s(2),
    hit3_i  => way_hit_s(3),
    data0_i => data0_s,
    data1_i => data1_s,
    data2_i => data2_s,

```

```

    data3_i => data3_s,
    hit_way_o => hit_way_s,
    data_o  => cache_data_s
);

```

```

U_DATA_WAY0 : entity work.cache_data_way_doc port map (clk_i => clk_i, set_i
=> set_s, we_i => way_we_s(0), data_i => data_in_s, data_o => data0_s);

```

```

U_DATA_WAY1 : entity work.cache_data_way_doc port map (clk_i => clk_i, set_i
=> set_s, we_i => way_we_s(1), data_i => data_in_s, data_o => data1_s);

```

```

U_DATA_WAY2 : entity work.cache_data_way_doc port map (clk_i => clk_i, set_i
=> set_s, we_i => way_we_s(2), data_i => data_in_s, data_o => data2_s);

```

```

U_DATA_WAY3 : entity work.cache_data_way_doc port map (clk_i => clk_i, set_i
=> set_s, we_i => way_we_s(3), data_i => data_in_s, data_o => data3_s);

```

```

U_TAG_WAY0 : entity work.cache_tag_way_doc port map (clk_i => clk_i, set_i
=> set_s, we_i => way_we_s(0), tag_i => tag_s, tag_o => tag0_s);

```

```

U_TAG_WAY1 : entity work.cache_tag_way_doc port map (clk_i => clk_i, set_i
=> set_s, we_i => way_we_s(1), tag_i => tag_s, tag_o => tag1_s);

```

```

U_TAG_WAY2 : entity work.cache_tag_way_doc port map (clk_i => clk_i, set_i
=> set_s, we_i => way_we_s(2), tag_i => tag_s, tag_o => tag2_s);

```

```

U_TAG_WAY3 : entity work.cache_tag_way_doc port map (clk_i => clk_i, set_i
=> set_s, we_i => way_we_s(3), tag_i => tag_s, tag_o => tag3_s);

```

```

U_VALID_WAY0 : entity work.cache_valid_way_doc port map (clk_i => clk_i,
rst_i => rst_i, set_i => set_s, we_i => way_we_s(0), valid_i => '1', valid_o =>
valid0_s);

```

```

U_VALID_WAY1 : entity work.cache_valid_way_doc port map (clk_i => clk_i,
rst_i => rst_i, set_i => set_s, we_i => way_we_s(1), valid_i => '1', valid_o =>
valid1_s);

```

```

U_VALID_WAY2 : entity work.cache_valid_way_doc port map (clk_i => clk_i,
rst_i => rst_i, set_i => set_s, we_i => way_we_s(2), valid_i => '1', valid_o =>
valid2_s);

```

```

U_VALID_WAY3 : entity work.cache_valid_way_doc port map (clk_i => clk_i,
rst_i => rst_i, set_i => set_s, we_i => way_we_s(3), valid_i => '1', valid_o =>
valid3_s);

```

```
U_AGE_WAY0 : entity work.cache_age_way_doc port map (clk_i => clk_i, rst_i
=> rst_i, set_i => set_s, we_i => way_we_s(0), inc_i => age_inc_s(0), age_i => "00",
age_o => age0_s);
```

```
U_AGE_WAY1 : entity work.cache_age_way_doc port map (clk_i => clk_i, rst_i
=> rst_i, set_i => set_s, we_i => way_we_s(1), inc_i => age_inc_s(1), age_i => "00",
age_o => age1_s);
```

```
U_AGE_WAY2 : entity work.cache_age_way_doc port map (clk_i => clk_i, rst_i
=> rst_i, set_i => set_s, we_i => way_we_s(2), inc_i => age_inc_s(2), age_i => "00",
age_o => age2_s);
```

```
U_AGE_WAY3 : entity work.cache_age_way_doc port map (clk_i => clk_i, rst_i
=> rst_i, set_i => set_s, we_i => way_we_s(3), inc_i => age_inc_s(3), age_i => "00",
age_o => age3_s);
```

```
U_TAG_COMPARE0 : entity work.cache_tag_compare_doc port map (req_tag_i
=> tag_s, way_tag_i => tag0_s, equal_o => tag_equal0_s);
```

```
U_TAG_COMPARE1 : entity work.cache_tag_compare_doc port map (req_tag_i
=> tag_s, way_tag_i => tag1_s, equal_o => tag_equal1_s);
```

```
U_TAG_COMPARE2 : entity work.cache_tag_compare_doc port map (req_tag_i
=> tag_s, way_tag_i => tag2_s, equal_o => tag_equal2_s);
```

```
U_TAG_COMPARE3 : entity work.cache_tag_compare_doc port map (req_tag_i
=> tag_s, way_tag_i => tag3_s, equal_o => tag_equal3_s);
```

```
U_HIT_MISS_LOGIC_AND_OR_NOT : entity work.cache_hit_miss_logic_doc
port map (
```

```
    tag_equal0_i => tag_equal0_s,
```

```
    tag_equal1_i => tag_equal1_s,
```

```
    tag_equal2_i => tag_equal2_s,
```

```
    tag_equal3_i => tag_equal3_s,
```

```
    valid0_i    => valid0_s,
```

```
    valid1_i    => valid1_s,
```

```
    valid2_i    => valid2_s,
```

```
    valid3_i    => valid3_s,
```

```
    way_hit_o   => way_hit_s,
```

```
    hit_o       => hit_any_s,
```

```
    miss_o      => miss_s
```

```
);
```

```
end architecture;
```

cpu_core.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity cpu_core is
  port (
    clk_i      : in  std_logic;
    rst_i      : in  std_logic;

    rom_addr_o  : out addr_t;
    rom_en_o    : out std_logic;
    rom_data_i  : in  word_t;

    cache_req_o : out std_logic;
    cache_we_o  : out std_logic;
    cache_addr_o : out addr_t;
    cache_wdata_o : out word_t;
    cache_rdata_i : in  word_t;
    cache_ready_i : in  std_logic;
    cache_hit_i  : in  std_logic;
    cache_miss_i : in  std_logic;

    bp_query_o  : out std_logic;
    bp_pc_query_o : out addr_t;
    bp_hist_i   : in  std_logic_vector(1 downto 0);
    bp_pred_i   : in  std_logic;
    bp_target_i : in  addr_t;
    bp_update_o : out std_logic;
    bp_pc_update_o : out addr_t;
    bp_hist_o   : out std_logic_vector(1 downto 0);
    bp_taken_o  : out std_logic;
    bp_target_o : out addr_t;

    halt_o      : out std_logic;

    dbg_state_o : out std_logic_vector(4 downto 0);
    dbg_pc_o    : out addr_t;
```

```

    dbg_ir0_o    : out word_t;
    dbg_ir1_o    : out word_t;
    dbg_r1_o     : out word_t;
    dbg_r2_o     : out word_t;
    dbg_r3_o     : out word_t;
    dbg_r4_o     : out word_t;
    dbg_r5_o     : out word_t;
    dbg_r6_o     : out word_t;
    dbg_r7_o     : out word_t;
    dbg_flags_o  : out std_logic_vector(3 downto 0);
    dbg_sp_o     : out unsigned(2 downto 0)
);
end entity;

```

architecture rtl of cpu_core is

```

    subtype state_code_t is std_logic_vector(4 downto 0);

```

```

    type state_t is (

```

```

        S_RESET,
        S_FETCH0_REQ,
        S_FETCH0_WAIT,
        S_FETCH0_LATCH,
        S_FETCH1_REQ,
        S_FETCH1_WAIT,
        S_FETCH1_LATCH,
        S_DECODE,
        S_CACHE_READ,
        S_CACHE_WRITE,
        S_ALU_SETUP_REG,
        S_ALU_WRITE,
        S_RF_WRITE_MEM,
        S_STACK_PUSH,
        S_STACK_POP,
        S_RF_WRITE_POP,
        S_BRANCH_JMP,
        S_BRANCH_JZ,
        S_FINISH,
        S_HALT
    );

```

);

```
signal state_r      : state_t := S_RESET;
signal pc_r         : addr_t := (others => '0');
signal ir0_r        : word_t := (others => '0');
signal ir1_r        : word_t := (others => '0');
signal cur_reg_r     : reg_idx_t := (others => '0');
signal cur_opcode_r : std_logic_vector(7 downto 0) := OP_HLT;
```

```
signal rom_addr_r   : addr_t := (others => '0');
signal rom_en_r     : std_logic := '0';
```

```
signal rf_we        : std_logic;
signal rf_wr_addr   : reg_idx_t;
signal rf_rd_a      : reg_idx_t;
signal rf_rd_b      : reg_idx_t;
signal rf_din       : word_t;
signal rf_a         : word_t;
signal rf_b         : word_t;
```

```
signal alu_a_r      : word_t := (others => '0');
signal alu_b_r      : word_t := (others => '0');
signal alu_op_r     : std_logic_vector(2 downto 0) := ALU_PASS_A;
signal alu_y        : word_t;
signal alu_z        : std_logic;
signal alu_s        : std_logic;
signal alu_c        : std_logic;
signal alu_o        : std_logic;
```

```
signal flag_we      : std_logic;
signal flag_z       : std_logic;
signal flag_s       : std_logic;
signal flag_c       : std_logic;
signal flag_o       : std_logic;
```

```
signal stack_push   : std_logic;
signal stack_pop     : std_logic;
signal stack_dout    : word_t;
signal stack_sp      : unsigned(2 downto 0);
```


signal stack_empty : std_logic;

signal stack_full : std_logic;

signal r1_dbg : word_t;

signal r2_dbg : word_t;

signal r3_dbg : word_t;

signal r4_dbg : word_t;

signal r5_dbg : word_t;

signal r6_dbg : word_t;

signal r7_dbg : word_t;

function state_code(s : state_t) return state_code_t is
begin

case s is

when S_RESET => return "00000";

when S_FETCH0_REQ => return "00001";

when S_FETCH0_WAIT => return "00010";

when S_FETCH0_LATCH => return "00011";

when S_FETCH1_REQ => return "00100";

when S_FETCH1_WAIT => return "00101";

when S_FETCH1_LATCH => return "00110";

when S_DECODE => return "00111";

when S_CACHE_READ => return "01000";

when S_CACHE_WRITE => return "01001";

when S_ALU_SETUP_REG=> return "01010";

when S_ALU_WRITE => return "01011";

when S_RF_WRITE_MEM => return "01100";

when S_STACK_PUSH => return "01101";

when S_STACK_POP => return "01110";

when S_RF_WRITE_POP => return "01111";

when S_BRANCH_JMP => return "10000";

when S_BRANCH_JZ => return "10001";

when S_FINISH => return "10010";

when S_HALT => return "11111";

end case;

end function;

begin

```

rom_addr_o <= rom_addr_r;
rom_en_o  <= rom_en_r;

rf_wr_addr <= cur_reg_r;
rf_rd_a   <= cur_reg_r;
rf_rd_b   <= (others => '0');
rf_we     <= '1' when state_r = S_ALU_WRITE or state_r = S_RF_WRITE_MEM
or state_r = S_RF_WRITE_POP else '0';
rf_din    <= alu_y when state_r = S_ALU_WRITE else
           cache_rdata_i when state_r = S_RF_WRITE_MEM else
           stack_dout;

flag_we <= '1' when state_r = S_ALU_WRITE else '0';

stack_push <= '1' when state_r = S_STACK_PUSH else '0';
stack_pop  <= '1' when state_r = S_STACK_POP else '0';

cache_req_o <= '1' when state_r = S_CACHE_READ or state_r =
S_CACHE_WRITE else '0';
cache_we_o  <= '1' when state_r = S_CACHE_WRITE else '0';
cache_addr_o <= ir1_r;
cache_wdata_o <= rf_a;

bp_query_o  <= '1' when state_r = S_FETCH0_REQ else '0';
bp_pc_query_o <= pc_r;
bp_update_o <= '1' when state_r = S_BRANCH_JZ else '0';
bp_pc_update_o <= pc_r;
bp_hist_o   <= bp_hist_i;
bp_taken_o  <= flag_z;
bp_target_o <= ir1_r;

halt_o     <= '1' when state_r = S_HALT else '0';
dbg_state_o <= state_code(state_r);
dbg_pc_o   <= pc_r;
dbg_ir0_o  <= ir0_r;
dbg_ir1_o  <= ir1_r;
dbg_r1_o   <= r1_dbg;
dbg_r2_o   <= r2_dbg;
dbg_r3_o   <= r3_dbg;

```

```

dbg_r4_o  <= r4_dbg;
dbg_r5_o  <= r5_dbg;
dbg_r6_o  <= r6_dbg;
dbg_r7_o  <= r7_dbg;
dbg_flags_o <= flag_z & flag_s & flag_c & flag_o;
dbg_sp_o  <= stack_sp;

```

U_RF : entity work.reg_file12x16_dbg

```

port map (
    clk_i    => clk_i,
    rst_i    => rst_i,
    we_i     => rf_we,
    wr_addr_i => rf_wr_addr,
    rd_addr_a_i => rf_rd_a,
    rd_addr_b_i => rf_rd_b,
    din_i    => rf_din,
    dout_a_o  => rf_a,
    dout_b_o  => rf_b,
    r0_o     => open,
    r1_o     => r1_dbg,
    r2_o     => r2_dbg,
    r3_o     => r3_dbg,
    r4_o     => r4_dbg,
    r5_o     => r5_dbg,
    r6_o     => r6_dbg,
    r7_o     => r7_dbg,
    r8_o     => open,
    r9_o     => open,
    r10_o    => open,
    r11_o    => open
);

```

U_ALU : entity work.alu_core

```

port map (
    a_i     => alu_a_r,
    b_i     => alu_b_r,
    flag_s_i => flag_s,

```

```

    op_i  => alu_op_r,
    y_o   => alu_y,
    z_o   => alu_z,
    s_o   => alu_s,
    c_o   => alu_c,
    o_o   => alu_o
);

```

U_FLAGS : entity work.flags_reg

```

port map (
    clk_i => clk_i,
    rst_i => rst_i,
    we_i  => flag_we,
    z_i   => alu_z,
    s_i   => alu_s,
    c_i   => alu_c,
    o_i   => alu_o,
    z_o   => flag_z,
    s_o   => flag_s,
    c_o   => flag_c,
    o_o   => flag_o
);

```

U_STACK : entity work.stack7x16

```

port map (
    clk_i  => clk_i,
    rst_i  => rst_i,
    push_i => stack_push,
    pop_i  => stack_pop,
    din_i  => rf_a,
    dout_o => stack_dout,
    sp_o   => stack_sp,
    empty_o => stack_empty,
    full_o => stack_full
);

```

process(clk_i)

```

variable opcode_v : std_logic_vector(7 downto 0);
variable next_pc : unsigned(15 downto 0);
begin
if rising_edge(clk_i) then
  if rst_i = '1' then
    state_r    <= S_RESET;
    pc_r       <= (others => '0');
    ir0_r      <= (others => '0');
    ir1_r      <= (others => '0');
    cur_reg_r  <= (others => '0');
    cur_opcode_r <= OP_HLT;
    rom_addr_r <= (others => '0');
    rom_en_r   <= '0';
    alu_a_r    <= (others => '0');
    alu_b_r    <= (others => '0');
    alu_op_r   <= ALU_PASS_A;
  else
    rom_en_r <= '1';
    next_pc := unsigned(pc_r) + 2;

    case state_r is
      when S_RESET =>
        pc_r    <= (others => '0');
        rom_addr_r <= (others => '0');
        state_r  <= S_FETCH0_REQ;

      when S_FETCH0_REQ =>
        rom_addr_r <= pc_r;
        state_r    <= S_FETCH0_WAIT;

      when S_FETCH0_WAIT =>
        state_r <= S_FETCH0_LATCH;

      when S_FETCH0_LATCH =>
        ir0_r    <= rom_data_i;
        rom_addr_r <= std_logic_vector(unsigned(pc_r) + 1);
        state_r  <= S_FETCH1_REQ;
    end case;
  end if;
end if;

```

```

when S_FETCH1_REQ =>
    state_r <= S_FETCH1_WAIT;

when S_FETCH1_WAIT =>
    state_r <= S_FETCH1_LATCH;

when S_FETCH1_LATCH =>
    ir1_r <= rom_data_i;
    state_r <= S_DECODE;

when S_DECODE =>
    opcode_v := ir0_r(15 downto 8);
    cur_opcode_r <= opcode_v;
    cur_reg_r <= unsigned(ir0_r(7 downto 4));
    if opcode_v = OP_MOV_MR or opcode_v = OP_OR or opcode_v =
OP_NOR then
        state_r <= S_CACHE_READ;
    elsif opcode_v = OP_MOV_RM then
        state_r <= S_CACHE_WRITE;
    elsif opcode_v = OP_SRA or opcode_v = OP_INCS then
        state_r <= S_ALU_SETUP_REG;
    elsif opcode_v = OP_PUSH then
        state_r <= S_STACK_PUSH;
    elsif opcode_v = OP_POP then
        state_r <= S_STACK_POP;
    elsif opcode_v = OP_JMP then
        state_r <= S_BRANCH_JMP;
    elsif opcode_v = OP_JZ then
        state_r <= S_BRANCH_JZ;
    elsif opcode_v = OP_HLT then
        state_r <= S_HALT;
    else
        state_r <= S_HALT;
    end if;

when S_CACHE_READ =>
    if cache_ready_i = '1' then
        if cur_opcode_r = OP_MOV_MR then

```

```

    state_r <= S_RF_WRITE_MEM;
else
    alu_a_r <= rf_a;
    alu_b_r <= cache_rdata_i;
    if cur_opcode_r = OP_OR then
        alu_op_r <= ALU_OR;
    else
        alu_op_r <= ALU_NOR;
    end if;
    state_r <= S_ALU_WRITE;
end if;
end if;

when S_CACHE_WRITE =>
    if cache_ready_i = '1' then
        state_r <= S_FINISH;
    end if;

when S_ALU_SETUP_REG =>
    alu_a_r <= rf_a;
    alu_b_r <= (others => '0');
    if cur_opcode_r = OP_SRA then
        alu_op_r <= ALU_SRA;
    else
        alu_op_r <= ALU_INCS;
    end if;
    state_r <= S_ALU_WRITE;

when S_ALU_WRITE =>
    state_r <= S_FINISH;

when S_RF_WRITE_MEM =>
    state_r <= S_FINISH;

when S_STACK_PUSH =>
    state_r <= S_FINISH;

when S_STACK_POP =>
    state_r <= S_RF_WRITE_POP;

```

```

when S_RF_WRITE_POP =>
    state_r <= S_FINISH;

when S_BRANCH_JMP =>
    pc_r <= ir1_r;
    state_r <= S_FETCH0_REQ;

when S_BRANCH_JZ =>
    if flag_z = '1' then
        pc_r <= ir1_r;
    else
        pc_r <= std_logic_vector(next_pc);
    end if;
    state_r <= S_FETCH0_REQ;

when S_FINISH =>
    pc_r <= std_logic_vector(next_pc);
    state_r <= S_FETCH0_REQ;

when S_HALT =>
    state_r <= S_HALT;
end case;
end if;
end if;
end process;

```

end architecture;

data_image_pkg.vhd

library ieee;

use ieee.std_logic_1164.all;

use ieee.numeric_std.all;

use work.microcomputer_pkg.all;

package data_image_pkg is

constant RAM_INIT : ram256_t := (

16#0A# => x"0000",

16#0B# => x"0000",

16#0C# => x"0000",


```

    16#20# => x"8001",
    16#21# => x"00F0",
    16#22# => x"0F0F",
    16#23# => x"0000",
    16#24# => x"1234",
    16#25# => x"FFFF",
    16#30# => x"0000",
    others => (others => '0')
);
end package;
dma_controller_3word.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity dma_controller_3word is
    generic (
        BASE_ADDR_G : addr_t := x"000A"
    );
    port (
        clk_i      : in std_logic;
        rst_i      : in std_logic;
        start_i    : in std_logic;
        grant_i     : in std_logic;
        dev_valid_i : in std_logic;
        dev_data_i  : in word_t;
        req_o      : out std_logic;
        busy_o     : out std_logic;
        done_o     : out std_logic;
        ram_we_o   : out std_logic;
        ram_addr_o  : out addr_t;
        ram_data_o  : out word_t
    );
end entity;

architecture rtl of dma_controller_3word is

```

```

type state_t is (IDLE, REQ_BUS, WRITE_WORD, FINISH);
signal state_r : state_t := IDLE;
signal index_r : unsigned(1 downto 0) := (others => '0');
signal done_r : std_logic := '0';
begin
    busy_o    <= '0' when state_r = IDLE else '1';
    req_o     <= '1' when state_r = REQ_BUS or state_r = WRITE_WORD else '0';
    ram_we_o  <= '1' when state_r = WRITE_WORD and grant_i = '1' and dev_valid_i
= '1' else '0';
    ram_addr_o <= std_logic_vector(unsigned(BASE_ADDR_G) + resize(index_r,
ADDR_W));
    ram_data_o <= dev_data_i;
    done_o    <= done_r;

    process(clk_i)
    begin
        if rising_edge(clk_i) then
            if rst_i = '1' then
                state_r <= IDLE;
                index_r <= (others => '0');
                done_r <= '0';
            else
                case state_r is
                    when IDLE =>
                        if start_i = '1' then
                            index_r <= (others => '0');
                            done_r <= '0';
                            state_r <= REQ_BUS;
                        end if;
                    when REQ_BUS =>
                        if grant_i = '1' then
                            state_r <= WRITE_WORD;
                        end if;
                    when WRITE_WORD =>
                        if grant_i = '1' and dev_valid_i = '1' then
                            if index_r = "10" then
                                state_r <= FINISH;
                            end if;
                        end if;
                    end case;
                end if;
            end process;

```

```

        else
            index_r <= index_r + 1;
        end if;
    end if;
when FINISH =>
    done_r <= '1';
    state_r <= IDLE;
end case;
end if;
end if;
end process;
end architecture;

```

flags_reg.vhd

```

library ieee;
use ieee.std_logic_1164.all;

entity flags_reg is
    port (
        clk_i : in std_logic;
        rst_i : in std_logic;
        we_i : in std_logic;
        z_i : in std_logic;
        s_i : in std_logic;
        c_i : in std_logic;
        o_i : in std_logic;
        z_o : out std_logic;
        s_o : out std_logic;
        c_o : out std_logic;
        o_o : out std_logic
    );
end entity;

architecture rtl of flags_reg is
    signal z_r, s_r, c_r, o_r : std_logic := '0';
begin
    z_o <= z_r;
    s_o <= s_r;

```

```

c_o <= c_r;
o_o <= o_r;

process(clk_i)
begin
  if rising_edge(clk_i) then
    if rst_i = '1' then
      z_r <= '0';
      s_r <= '0';
      c_r <= '0';
      o_r <= '0';
    elsif we_i = '1' then
      z_r <= z_i;
      s_r <= s_i;
      c_r <= c_i;
      o_r <= o_i;
    end if;
  end if;
end process;
end architecture;
microcomputer_debug_top.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity microcomputer_debug_top is
  port (
    CLK          : in std_logic;
    RESET        : in std_logic;
    DMA_START    : in std_logic;
    DMA_VALID    : in std_logic;
    DMA_DATA     : in word_t;

    HALT         : out std_logic;
    DMA_DONE     : out std_logic;

    DBG_STATE    : out std_logic_vector(4 downto 0);

```

```

DBG_PC      : out addr_t;
DBG_IR0     : out word_t;
DBG_IR1     : out word_t;
DBG_R1      : out word_t;
DBG_R2      : out word_t;
DBG_R3      : out word_t;
DBG_R4      : out word_t;
DBG_R5      : out word_t;
DBG_R6      : out word_t;
DBG_R7      : out word_t;
DBG_FLAGS   : out std_logic_vector(3 downto 0);
DBG_SP      : out unsigned(2 downto 0);
DBG_CACHE_HIT : out std_logic;
DBG_CACHE_MISS : out std_logic;
DBG_REQ_CPU  : out std_logic;
DBG_REQ_DMA  : out std_logic;
DBG_GNT_CPU  : out std_logic;
DBG_GNT_DMA  : out std_logic;
DBG_RAM_WE   : out std_logic;
DBG_RAM_ADDR : out addr_t;
DBG_RAM_WDATA : out word_t;
DBG_RAM_RDATA : out word_t;
DBG_BP_HIST  : out std_logic_vector(1 downto 0);
DBG_BP_PRED  : out std_logic;
DBG_BP_TARGET : out addr_t
);
end entity;

```

architecture structural of microcomputer_debug_top is
begin

```

U_SYSTEM : entity work.system_core
port map (
    clk_i      => CLK,
    rst_i      => RESET,
    dma_start_i  => DMA_START,
    dma_valid_i  => DMA_VALID,
    dma_data_i   => DMA_DATA,

```

```

halt_o      => HALT,
dma_done_o   => DMA_DONE,
dbg_state_o  => DBG_STATE,
dbg_pc_o     => DBG_PC,
dbg_ir0_o    => DBG_IR0,
dbg_ir1_o    => DBG_IR1,
dbg_r1_o     => DBG_R1,
dbg_r2_o     => DBG_R2,
dbg_r3_o     => DBG_R3,
dbg_r4_o     => DBG_R4,
dbg_r5_o     => DBG_R5,
dbg_r6_o     => DBG_R6,
dbg_r7_o     => DBG_R7,
dbg_flags_o  => DBG_FLAGS,
dbg_sp_o     => DBG_SP,
dbg_cache_hit_o => DBG_CACHE_HIT,
dbg_cache_miss_o => DBG_CACHE_MISS,
dbg_req_cpu_o  => DBG_REQ_CPU,
dbg_req_dma_o  => DBG_REQ_DMA,
dbg_gnt_cpu_o  => DBG_GNT_CPU,
dbg_gnt_dma_o  => DBG_GNT_DMA,
dbg_ram_we_o   => DBG_RAM_WE,
dbg_ram_addr_o => DBG_RAM_ADDR,
dbg_ram_wdata_o => DBG_RAM_WDATA,
dbg_ram_rdata_o => DBG_RAM_RDATA,
dbg_bp_hist_o  => DBG_BP_HIST,
dbg_bp_pred_o  => DBG_BP_PRED,
dbg_bp_target_o => DBG_BP_TARGET
);
end architecture;

microcomputer_pkg.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

package microcomputer_pkg is
    constant DATA_W : positive := 16;

```

```

constant ADDR_W    : positive := 16;
constant REG_COUNT  : positive := 12;
constant STACK_DEPTH : positive := 7;
constant CACHE_SETS : positive := 16;
constant CACHE_WAYS : positive := 4;

subtype word_t  is std_logic_vector(DATA_W-1 downto 0);
subtype addr_t  is std_logic_vector(ADDR_W-1 downto 0);
subtype reg_idx_t is unsigned(3 downto 0);

type ram256_t is array (0 to 255) of word_t;

constant ALU_PASS_A : std_logic_vector(2 downto 0) := "000";
constant ALU_OR      : std_logic_vector(2 downto 0) := "001";
constant ALU_NOR     : std_logic_vector(2 downto 0) := "010";
constant ALU_SRA     : std_logic_vector(2 downto 0) := "011";
constant ALU_INCS    : std_logic_vector(2 downto 0) := "100";

constant OP_HLT      : std_logic_vector(7 downto 0) := x"00";
constant OP_MOV_MR   : std_logic_vector(7 downto 0) := x"01";
constant OP_MOV_RM   : std_logic_vector(7 downto 0) := x"02";
constant OP_OR       : std_logic_vector(7 downto 0) := x"03";
constant OP_NOR      : std_logic_vector(7 downto 0) := x"04";
constant OP_SRA      : std_logic_vector(7 downto 0) := x"05";
constant OP_INCS     : std_logic_vector(7 downto 0) := x"06";
constant OP_PUSH     : std_logic_vector(7 downto 0) := x"07";
constant OP_POP      : std_logic_vector(7 downto 0) := x"08";
constant OP_JMP      : std_logic_vector(7 downto 0) := x"09";
constant OP_JZ       : std_logic_vector(7 downto 0) := x"0A";

function sat_inc2(v : unsigned(1 downto 0)) return unsigned;
end package;

package body microcomputer_pkg is
function sat_inc2(v : unsigned(1 downto 0)) return unsigned is
begin
    if v = "11" then
        return v;
    else

```

```

        return v + 1;
    end if;
end function;
end package body;
microcomputer_top.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

```

```

entity microcomputer_top is
    port (
        CLK          : in std_logic;
        RESET         : in std_logic;
        DMA_START     : in std_logic;
        DMA_VALID     : in std_logic;
        DMA_DATA      : in word_t;

        HALT          : out std_logic;
        DMA_DONE       : out std_logic
    );
end entity;

```

```

architecture structural of microcomputer_top is
begin

```

```

    U_SYSTEM : entity work.system_core
    port map (
        clk_i      => CLK,
        rst_i      => RESET,
        dma_start_i => DMA_START,
        dma_valid_i => DMA_VALID,
        dma_data_i  => DMA_DATA,
        halt_o      => HALT,
        dma_done_o  => DMA_DONE,
        dbg_state_o => open,
        dbg_pc_o    => open,
        dbg_ir0_o   => open,

```



```

    dbg_ir1_o    => open,
    dbg_r1_o     => open,
    dbg_r2_o     => open,
    dbg_r3_o     => open,
    dbg_r4_o     => open,
    dbg_r5_o     => open,
    dbg_r6_o     => open,
    dbg_r7_o     => open,
    dbg_flags_o  => open,
    dbg_sp_o     => open,
    dbg_cache_hit_o => open,
    dbg_cache_miss_o => open,
    dbg_req_cpu_o  => open,
    dbg_req_dma_o  => open,
    dbg_gnt_cpu_o  => open,
    dbg_gnt_dma_o  => open,
    dbg_ram_we_o   => open,
    dbg_ram_addr_o => open,
    dbg_ram_wdata_o => open,
    dbg_ram_rdata_o => open,
    dbg_bp_hist_o  => open,
    dbg_bp_pred_o  => open,
    dbg_bp_target_o => open
);
end architecture;

program_image_pkg.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

package program_image_pkg is
    constant ROM_INIT : ram256_t := (
        16#00# => x"0110", -- MOV 0020h, R1
        16#01# => x"0020",
        16#02# => x"0510", -- SRA R1
        16#03# => x"0000",

```

```

16#04# => x"0610", -- INCS R1
16#05# => x"0000",
16#06# => x"0710", -- PUSH R1
16#07# => x"0000",
16#08# => x"0120", -- MOV 0021h, R2
16#09# => x"0021",
16#0A# => x"0320", -- OR R2, 0022h
16#0B# => x"0022",
16#0C# => x"0420", -- NOR R2, 0023h
16#0D# => x"0023",
16#0E# => x"0830", -- POP R3
16#0F# => x"0000",
16#10# => x"0230", -- MOV R3, 0030h
16#11# => x"0030",
16#12# => x"0140", -- MOV 0025h, R4
16#13# => x"0025",
16#14# => x"0440", -- NOR R4, 0025h
16#15# => x"0025",
16#16# => x"0A00", -- JZ 001Ah
16#17# => x"001A",
16#18# => x"0150", -- MOV 0024h, R5 (must be skipped)
16#19# => x"0024",
16#1A# => x"0900", -- JMP 001Eh
16#1B# => x"001E",
16#1C# => x"0160", -- MOV 0024h, R6 (must be skipped)
16#1D# => x"0024",
16#1E# => x"0170", -- MOV 000Ah, R7
16#1F# => x"000A",
16#20# => x"0000", -- HLT
16#21# => x"0000",
others => (others => '0')
);
end package;
ram_sync.vhd
library ieee;
use ieee.std_logic_1164.all;

```

```

use ieee.numeric_std.all;
use work.microcomputer_pkg.all;
use work.data_image_pkg.all;

entity ram_sync is
  port (
    clk_i : in std_logic;
    en_i  : in std_logic;
    we_i  : in std_logic;
    addr_i : in addr_t;
    din_i : in word_t;
    dout_o : out word_t
  );
end entity;

architecture rtl of ram_sync is
  signal mem : ram256_t := RAM_INIT;
  signal q_r : word_t := (others => '0');
begin
  dout_o <= q_r;

  process(clk_i)
    variable idx : integer;
  begin
    if rising_edge(clk_i) then
      if en_i = '1' then
        idx := to_integer(unsigned(addr_i(7 downto 0)));
        if we_i = '1' then
          mem(idx) <= din_i;
        end if;
        q_r <= mem(idx);
      end if;
    end if;
  end process;
end architecture;

reg_file12x16.vhd
library ieee;

```

```
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;
```

entity reg_file12x16 is

```
port (
    clk_i    : in std_logic;
    rst_i    : in std_logic;
    we_i     : in std_logic;
    wr_addr_i : in reg_idx_t;
    rd_addr_a_i : in reg_idx_t;
    rd_addr_b_i : in reg_idx_t;
    din_i    : in word_t;
    dout_a_o  : out word_t;
    dout_b_o  : out word_t
);
```

end entity;

architecture rtl of reg_file12x16 is

```
type reg_array_t is array (0 to REG_COUNT-1) of word_t;
signal regs : reg_array_t := (others => (others => '0'));
```

function read_reg(mem : reg_array_t; idx : reg_idx_t) return word_t is

```
begin
    if to_integer(idx) < REG_COUNT then
        return mem(to_integer(idx));
    else
        return (others => '0');
    end if;
```

end function;

begin

```
dout_a_o <= read_reg(regs, rd_addr_a_i);
dout_b_o <= read_reg(regs, rd_addr_b_i);
```

process(clk_i)

```
begin
    if rising_edge(clk_i) then
        if rst_i = '1' then
```

```

        regs <= (others => (others => '0'));
    elsif we_i = '1' and to_integer(wr_addr_i) < REG_COUNT then
        regs(to_integer(wr_addr_i)) <= din_i;
    end if;
end if;
end process;
end architecture;
reg_file12x16_dbg.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity reg_file12x16_dbg is
port (
    clk_i    : in std_logic;
    rst_i    : in std_logic;
    we_i     : in std_logic;
    wr_addr_i : in reg_idx_t;
    rd_addr_a_i : in reg_idx_t;
    rd_addr_b_i : in reg_idx_t;
    din_i    : in word_t;
    dout_a_o  : out word_t;
    dout_b_o  : out word_t;
    r0_o     : out word_t;
    r1_o     : out word_t;
    r2_o     : out word_t;
    r3_o     : out word_t;
    r4_o     : out word_t;
    r5_o     : out word_t;
    r6_o     : out word_t;
    r7_o     : out word_t;
    r8_o     : out word_t;
    r9_o     : out word_t;
    r10_o    : out word_t;
    r11_o    : out word_t
);

```

end entity;

architecture rtl of reg_file12x16_dbg is

type reg_array_t is array (0 to REG_COUNT-1) of word_t;

signal regs : reg_array_t := (others => (others => '0'));

function read_reg(mem : reg_array_t; idx : reg_idx_t) return word_t is

begin

if to_integer(idx) < REG_COUNT then

return mem(to_integer(idx));

else

return (others => '0');

end if;

end function;

begin

dout_a_o <= read_reg(regs, rd_addr_a_i);

dout_b_o <= read_reg(regs, rd_addr_b_i);

r0_o <= regs(0);

r1_o <= regs(1);

r2_o <= regs(2);

r3_o <= regs(3);

r4_o <= regs(4);

r5_o <= regs(5);

r6_o <= regs(6);

r7_o <= regs(7);

r8_o <= regs(8);

r9_o <= regs(9);

r10_o <= regs(10);

r11_o <= regs(11);

process(clk_i)

begin

if rising_edge(clk_i) then

if rst_i = '1' then

regs <= (others => (others => '0'));

elsif we_i = '1' and to_integer(wr_addr_i) < REG_COUNT then

regs(to_integer(wr_addr_i)) <= din_i;

```

        end if;
    end if;
end process;
end architecture;
rom_sync.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;
use work.program_image_pkg.all;

entity rom_sync is
    port (
        clk_i : in std_logic;
        en_i   : in std_logic;
        addr_i : in addr_t;
        data_o : out word_t
    );
end entity;

architecture rtl of rom_sync is
    signal q_r : word_t := (others => '0');
begin
    data_o <= q_r;

    process(clk_i)
        variable idx : integer;
    begin
        if rising_edge(clk_i) then
            if en_i = '1' then
                idx := to_integer(unsigned(addr_i(7 downto 0)));
                q_r <= ROM_INIT(idx);
            end if;
        end if;
    end process;
end architecture;
stack7x16.vhd

```

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity stack7x16 is
  port (
    clk_i  : in std_logic;
    rst_i  : in std_logic;
    push_i : in std_logic;
    pop_i  : in std_logic;
    din_i  : in word_t;
    dout_o : out word_t;
    sp_o   : out unsigned(2 downto 0);
    empty_o : out std_logic;
    full_o  : out std_logic
  );
end entity;

```

architecture rtl of stack7x16 is

```

  type stack_mem_t is array (0 to STACK_DEPTH-1) of word_t;
  signal mem      : stack_mem_t := (others => (others => '0'));
  signal sp_r     : unsigned(2 downto 0) := to_unsigned(STACK_DEPTH, 3);
  signal dout_r  : word_t := (others => '0');
begin
  dout_o <= dout_r;
  sp_o   <= sp_r;
  empty_o <= '1' when sp_r = to_unsigned(STACK_DEPTH, 3) else '0';
  full_o  <= '1' when sp_r = to_unsigned(0, 3) else '0';

  process(clk_i)
    variable sp_i : integer;
  begin
    if rising_edge(clk_i) then
      if rst_i = '1' then
        mem <= (others => (others => '0'));
        sp_r <= to_unsigned(STACK_DEPTH, 3);

```



```

    dout_r <= (others => '0');
else
    sp_i := to_integer(sp_r);

    if push_i = '1' and pop_i = '0' then
        if sp_i > 0 then
            mem(sp_i - 1) <= din_i;
            sp_r <= to_unsigned(sp_i - 1, 3);
        end if;
    elsif pop_i = '1' and push_i = '0' then
        if sp_i < STACK_DEPTH then
            dout_r <= mem(sp_i);
            sp_r <= to_unsigned(sp_i + 1, 3);
        end if;
    end if;
end if;
end if;
end process;
end architecture;
system_core.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity system_core is
port (
    clk_i      : in std_logic;
    rst_i      : in std_logic;
    dma_start_i : in std_logic;
    dma_valid_i : in std_logic;
    dma_data_i  : in word_t;

    halt_o      : out std_logic;
    dma_done_o   : out std_logic;

    dbg_state_o  : out std_logic_vector(4 downto 0);
    dbg_pc_o     : out addr_t;

```

```

    dbg_ir0_o    : out word_t;
    dbg_ir1_o    : out word_t;
    dbg_r1_o     : out word_t;
    dbg_r2_o     : out word_t;
    dbg_r3_o     : out word_t;
    dbg_r4_o     : out word_t;
    dbg_r5_o     : out word_t;
    dbg_r6_o     : out word_t;
    dbg_r7_o     : out word_t;
    dbg_flags_o  : out std_logic_vector(3 downto 0);
    dbg_sp_o     : out unsigned(2 downto 0);

    dbg_cache_hit_o : out std_logic;
    dbg_cache_miss_o : out std_logic;
    dbg_req_cpu_o   : out std_logic;
    dbg_req_dma_o   : out std_logic;
    dbg_gnt_cpu_o   : out std_logic;
    dbg_gnt_dma_o   : out std_logic;
    dbg_ram_we_o    : out std_logic;
    dbg_ram_addr_o  : out addr_t;
    dbg_ram_wdata_o : out word_t;
    dbg_ram_rdata_o : out word_t;
    dbg_bp_hist_o   : out std_logic_vector(1 downto 0);
    dbg_bp_pred_o   : out std_logic;
    dbg_bp_target_o : out addr_t
);
end entity;

```

architecture structural of system_core is

```

    signal rom_addr    : addr_t;
    signal rom_en      : std_logic;
    signal rom_data    : word_t;

    signal cache_req   : std_logic;
    signal cache_we    : std_logic;
    signal cache_addr  : addr_t;
    signal cache_wdata : word_t;

```

```

signal cache_rdata : word_t;
signal cache_ready : std_logic;
signal cache_hit   : std_logic;
signal cache_miss  : std_logic;

signal cache_ram_req : std_logic;
signal cache_ram_we  : std_logic;
signal cache_ram_addr : addr_t;
signal cache_ram_wdata : word_t;

signal dma_req      : std_logic;
signal dma_grant    : std_logic;
signal cpu_grant    : std_logic;
signal dma_done     : std_logic;
signal dma_we       : std_logic;
signal dma_addr     : addr_t;
signal dma_wdata    : word_t;

signal ram_en       : std_logic;
signal ram_we       : std_logic;
signal ram_addr     : addr_t;
signal ram_wdata    : word_t;
signal ram_rdata    : word_t;

signal bp_query     : std_logic;
signal bp_pc_query  : addr_t;
signal bp_hist      : std_logic_vector(1 downto 0);
signal bp_pred      : std_logic;
signal bp_target    : addr_t;
signal bp_update    : std_logic;
signal bp_pc_update : addr_t;
signal bp_hist_upd  : std_logic_vector(1 downto 0);
signal bp_taken     : std_logic;
signal bp_tgt_upd   : addr_t;
begin
    ram_en  <= (cache_ram_req and cpu_grant) or (dma_we and dma_grant);
    ram_we  <= dma_we when dma_grant = '1' else cache_ram_we;
    ram_addr <= dma_addr when dma_grant = '1' else cache_ram_addr;

```

```

ram_wdata <= dma_wdata when dma_grant = '1' else cache_ram_wdata;

dma_done_o    <= dma_done;
dbg_cache_hit_o <= cache_hit;
dbg_cache_miss_o <= cache_miss;
dbg_req_cpu_o  <= cache_ram_req;
dbg_req_dma_o  <= dma_req;
dbg_gnt_cpu_o  <= cpu_grant;
dbg_gnt_dma_o  <= dma_grant;
dbg_ram_we_o   <= ram_we;
dbg_ram_addr_o <= ram_addr;
dbg_ram_wdata_o <= ram_wdata;
dbg_ram_rdata_o <= ram_rdata;
dbg_bp_hist_o  <= bp_hist;
dbg_bp_pred_o  <= bp_pred;
dbg_bp_target_o <= bp_target;

U_CPU : entity work.cpu_core
port map (
    clk_i      => clk_i,
    rst_i      => rst_i,
    rom_addr_o  => rom_addr,
    rom_en_o    => rom_en,
    rom_data_i  => rom_data,
    cache_req_o => cache_req,
    cache_we_o  => cache_we,
    cache_addr_o => cache_addr,
    cache_wdata_o => cache_wdata,
    cache_rdata_i => cache_rdata,
    cache_ready_i => cache_ready,
    cache_hit_i  => cache_hit,
    cache_miss_i => cache_miss,
    bp_query_o   => bp_query,
    bp_pc_query_o => bp_pc_query,
    bp_hist_i    => bp_hist,
    bp_pred_i    => bp_pred,
    bp_target_i  => bp_target,

```

```

bp_update_o  => bp_update,
bp_pc_update_o => bp_pc_update,
bp_hist_o    => bp_hist_upd,
bp_taken_o   => bp_taken,
bp_target_o  => bp_tgt_upd,
halt_o       => halt_o,
dbg_state_o  => dbg_state_o,
dbg_pc_o     => dbg_pc_o,
dbg_ir0_o    => dbg_ir0_o,
dbg_ir1_o    => dbg_ir1_o,
dbg_r1_o     => dbg_r1_o,
dbg_r2_o     => dbg_r2_o,
dbg_r3_o     => dbg_r3_o,
dbg_r4_o     => dbg_r4_o,
dbg_r5_o     => dbg_r5_o,
dbg_r6_o     => dbg_r6_o,
dbg_r7_o     => dbg_r7_o,
dbg_flags_o  => dbg_flags_o,
dbg_sp_o     => dbg_sp_o
);

```

U_ROM : entity work.rom_sync

```

port map (
    clk_i  => clk_i,
    en_i   => rom_en,
    addr_i => rom_addr,
    data_o => rom_data
);

```

U_CACHE : entity work.cache_4way_age

```

port map (
    clk_i    => clk_i,
    rst_i    => rst_i,
    cpu_req_i => cache_req,
    cpu_we_i  => cache_we,
    cpu_addr_i => cache_addr,
    cpu_wdata_i => cache_wdata,

```

```

    cpu_rdata_o => cache_rdata,
    cpu_ready_o => cache_ready,
    hit_o      => cache_hit,
    miss_o     => cache_miss,
    ram_req_o  => cache_ram_req,
    ram_we_o   => cache_ram_we,
    ram_addr_o => cache_ram_addr,
    ram_wdata_o => cache_ram_wdata,
    ram_rdata_i => ram_rdata,
    ram_grant_i => cpu_grant
);

```

U_RAM : entity work.ram_sync

```

port map (
    clk_i  => clk_i,
    en_i   => ram_en,
    we_i   => ram_we,
    addr_i => ram_addr,
    din_i  => ram_wdata,
    dout_o => ram_rdata
);

```

U_ARB : entity work.bus_arbiter_2master

```

port map (
    req_cpu_i  => cache_ram_req,
    req_dma_i  => dma_req,
    grant_cpu_o => cpu_grant,
    grant_dma_o => dma_grant
);

```

U_DMA : entity work.dma_controller_3word

```

port map (
    clk_i    => clk_i,
    rst_i    => rst_i,
    start_i  => dma_start_i,
    grant_i  => dma_grant,
    dev_valid_i => dma_valid_i,

```

```

    dev_data_i => dma_data_i,
    req_o      => dma_req,
    busy_o     => open,
    done_o     => dma_done,
    ram_we_o   => dma_we,
    ram_addr_o => dma_addr,
    ram_data_o => dma_wdata
);

```

U_BP : entity work.branch_predictor

```

port map (
    clk_i      => clk_i,
    rst_i      => rst_i,
    query_valid_i => bp_query,
    pc_query_i  => bp_pc_query,
    hist_o     => bp_hist,
    pred_taken_o => bp_pred,
    pred_target_o => bp_target,
    update_valid_i => bp_update,
    pc_update_i  => bp_pc_update,
    hist_i     => bp_hist_upd,
    actual_taken_i => bp_taken,
    actual_tgt_i  => bp_tgt_upd
);

```

end architecture;

Е.2 Проверочные стенды

tb_alu_core.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity tb_alu_core is
end entity;

architecture sim of tb_alu_core is
    signal a      : word_t := (others => '0');
    signal b      : word_t := (others => '0');
    signal sflag   : std_logic := '0';
    signal op      : std_logic_vector(2 downto 0) := ALU_PASS_A;
    signal y       : word_t;
    signal z, s, c, o : std_logic;
begin
    DUT : entity work.alu_core
        port map (
            a_i => a,
            b_i => b,
            flag_s_i => sflag,
            op_i => op,
            y_o => y,
            z_o => z,
            s_o => s,
            c_o => c,
            o_o => o
        );

    stimulus : process
    begin
        a <= x"8001";
        b <= x"00F0";
        op <= ALU_SRA;
        wait for 10 ns;
```



```

assert y = x"C000" and c = '1' and s = '1'
    report "SRA 8001h must produce C000h, C=1, S=1" severity failure;

a <= y;
sflag <= '1';
op <= ALU_INCS;
wait for 10 ns;
assert y = x"C001"
    report "INCS with S=1 must increment C000h to C001h" severity failure;

a <= x"00F0";
b <= x"0F0F";
op <= ALU_OR;
wait for 10 ns;
assert y = x"0FFF"
    report "OR must produce 0FFFh" severity failure;

a <= x"0FFF";
b <= x"0FFF";
op <= ALU_NOR;
wait for 10 ns;
assert y = x"F000"
    report "NOR must produce F000h" severity failure;

sflag <= '0';
a <= x"1234";
op <= ALU_INCS;
wait for 10 ns;
assert y = x"1234"
    report "INCS with S=0 must keep source value" severity failure;

assert false report "tb_alu_core: TEST PASSED" severity note;
wait;
end process;
end architecture;
tb_mem_dma.vhd
library ieee;
use ieee.std_logic_1164.all;

```

```
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;
```

```
entity tb_mem_dma is
end entity;
```

```
architecture sim of tb_mem_dma is
```

```
    signal clk      : std_logic := '0';
    signal rst      : std_logic := '1';
    signal start    : std_logic := '0';
    signal dev_valid: std_logic := '0';
    signal dev_data : word_t    := (others => '0');
    signal dma_req  : std_logic;
    signal dma_gnt  : std_logic;
    signal dma_done : std_logic;
    signal ram_we   : std_logic;
    signal ram_addr : addr_t;
    signal ram_din  : word_t;
    signal ram_dout : word_t;
```

```
begin
```

```
    clk <= not clk after 5 ns;
    dma_gnt <= dma_req;
```

```
U_DMA : entity work.dma_controller_3word
```

```
port map (
```

```
    clk_i => clk,
    rst_i => rst,
    start_i => start,
    grant_i => dma_gnt,
    dev_valid_i => dev_valid,
    dev_data_i => dev_data,
    req_o => dma_req,
    busy_o => open,
    done_o => dma_done,
    ram_we_o => ram_we,
    ram_addr_o => ram_addr,
    ram_data_o => ram_din
```

);

U_RAM : entity work.ram_sync

port map (

clk_i => clk,

en_i => '1',

we_i => ram_we,

addr_i => ram_addr,

din_i => ram_din,

dout_o => ram_dout

);

monitor : process(clk)

variable write_count : integer := 0;

begin

if rising_edge(clk) then

if rst = '1' then

write_count := 0;

elsif ram_we = '1' then

case write_count is

when 0 =>

assert ram_addr = x"000A" and ram_din = x"1111"

report "DMA first write must target 000Ah with 1111h" severity failure;

when 1 =>

assert ram_addr = x"000B" and ram_din = x"2222"

report "DMA second write must target 000Bh with 2222h" severity failure;

when 2 =>

assert ram_addr = x"000C" and ram_din = x"3333"

report "DMA third write must target 000Ch with 3333h" severity failure;

when others =>

assert false report "Unexpected extra DMA write" severity failure;

end case;

write_count := write_count + 1;

end if;

end if;

end process;

```

stimulus : process
begin
    wait for 30 ns;
    rst <= '0';

    wait until rising_edge(clk);
    start <= '1';
    dev_valid <= '1';
    dev_data <= x"1111";
    wait until rising_edge(clk);
    start <= '0';

    wait until rising_edge(clk);
    dev_data <= x"1111";
    wait until rising_edge(clk);
    dev_data <= x"2222";
    wait until rising_edge(clk);
    dev_data <= x"3333";
    wait until rising_edge(clk);
    dev_valid <= '0';
    wait until rising_edge(clk);
    wait for 1 ns;
    assert dma_done = '1'
        report "DMA must assert done after three words" severity failure;

    assert false report "tb_mem_dma: TEST PASSED" severity note;
    wait;
end process;
end architecture;
tb_stack7x16.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity tb_stack7x16 is
end entity;

```

architecture sim of tb_stack7x16 is

```
signal clk  : std_logic := '0';
signal rst  : std_logic := '1';
signal push : std_logic := '0';
signal pop  : std_logic := '0';
signal din  : word_t := (others => '0');
signal dout : word_t;
signal sp   : unsigned(2 downto 0);
signal empty : std_logic;
signal full  : std_logic;
```

begin

```
clk <= not clk after 5 ns;
```

DUT : entity work.stack7x16

```
port map (
  clk_i => clk,
  rst_i => rst,
  push_i => push,
  pop_i => pop,
  din_i => din,
  dout_o => dout,
  sp_o => sp,
  empty_o => empty,
  full_o => full
);
```

stimulus : process

begin

```
wait for 12 ns;
rst <= '0';
wait for 1 ns;
assert sp = to_unsigned(7, 3) and empty = '1' and full = '0'
  report "After reset stack must be empty with SP=7" severity failure;
```

```
din <= x"0002"; push <= '1'; wait for 10 ns; push <= '0';
din <= x"0006"; push <= '1'; wait for 10 ns; push <= '0';
din <= x"0007"; push <= '1'; wait for 10 ns; push <= '0';
```

```

din <= x"0078"; push <= '1'; wait for 10 ns; push <= '0';
din <= x"03E0"; push <= '1'; wait for 10 ns; push <= '0';
wait for 1 ns;
assert sp = to_unsigned(2, 3)
    report "After five pushes SP must be 2" severity failure;

pop <= '1'; wait for 10 ns; pop <= '0';
wait for 1 ns;
assert dout = x"03E0"
    report "First pop must return last pushed value 03E0h" severity failure;
pop <= '1'; wait for 10 ns; pop <= '0';
pop <= '1'; wait for 10 ns; pop <= '0';
pop <= '1'; wait for 10 ns; pop <= '0';
pop <= '1'; wait for 10 ns; pop <= '0';
wait for 1 ns;
assert sp = to_unsigned(7, 3) and empty = '1'
    report "After five pops stack must be empty with SP=7" severity failure;

assert false report "tb_stack7x16: TEST PASSED" severity note;
wait;
end process;
end architecture;
tb_system.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.microcomputer_pkg.all;

entity tb_system is
end entity;

architecture sim of tb_system is
    signal clk      : std_logic := '0';
    signal rst      : std_logic := '1';
    signal dma_start : std_logic := '0';
    signal dma_valid  : std_logic := '0';
    signal dma_data   : word_t := (others => '0');
    signal halt       : std_logic;

```

```

signal dma_done    : std_logic;

signal dbg_state   : std_logic_vector(4 downto 0);
signal dbg_pc      : addr_t;
signal dbg_ir0     : word_t;
signal dbg_ir1     : word_t;
signal dbg_r1      : word_t;
signal dbg_r2      : word_t;
signal dbg_r3      : word_t;
signal dbg_r4      : word_t;
signal dbg_r5      : word_t;
signal dbg_r6      : word_t;
signal dbg_r7      : word_t;
signal dbg_flags   : std_logic_vector(3 downto 0);
signal dbg_sp      : unsigned(2 downto 0);
signal cache_hit   : std_logic;
signal cache_miss  : std_logic;
signal req_cpu     : std_logic;
signal req_dma     : std_logic;
signal gnt_cpu     : std_logic;
signal gnt_dma     : std_logic;
signal ram_we      : std_logic;
signal ram_addr    : addr_t;
signal ram_wdata   : word_t;
signal ram_rdata   : word_t;
signal bp_hist     : std_logic_vector(1 downto 0);
signal bp_pred     : std_logic;
signal bp_target   : addr_t;
begin
    clk <= not clk after 5 ns;

    UUT : entity work.system_core
    port map (
        clk_i        => clk,
        rst_i        => rst,
        dma_start_i   => dma_start,
        dma_valid_i   => dma_valid,

```

```

dma_data_i    => dma_data,
halt_o        => halt,
dma_done_o    => dma_done,
dbg_state_o   => dbg_state,
dbg_pc_o      => dbg_pc,
dbg_ir0_o     => dbg_ir0,
dbg_ir1_o     => dbg_ir1,
dbg_r1_o      => dbg_r1,
dbg_r2_o      => dbg_r2,
dbg_r3_o      => dbg_r3,
dbg_r4_o      => dbg_r4,
dbg_r5_o      => dbg_r5,
dbg_r6_o      => dbg_r6,
dbg_r7_o      => dbg_r7,
dbg_flags_o   => dbg_flags,
dbg_sp_o      => dbg_sp,
dbg_cache_hit_o => cache_hit,
dbg_cache_miss_o => cache_miss,
dbg_req_cpu_o  => req_cpu,
dbg_req_dma_o  => req_dma,
dbg_gnt_cpu_o  => gnt_cpu,
dbg_gnt_dma_o  => gnt_dma,
dbg_ram_we_o   => ram_we,
dbg_ram_addr_o => ram_addr,
dbg_ram_wdata_o => ram_wdata,
dbg_ram_rdata_o => ram_rdata,
dbg_bp_hist_o  => bp_hist,
dbg_bp_pred_o  => bp_pred,
dbg_bp_target_o => bp_target
);

```

stim : process

begin

rst <= '1';

wait for 30 ns;

rst <= '0';


```

wait until rising_edge(clk);
wait until rising_edge(clk);
dma_start <= '1';
dma_valid <= '1';
dma_data <= x"1111";
wait until rising_edge(clk);
dma_start <= '0';

wait until rising_edge(clk);
dma_data <= x"1111";
wait until rising_edge(clk);
dma_data <= x"2222";
wait until rising_edge(clk);
dma_data <= x"3333";
wait until rising_edge(clk);
dma_valid <= '0';

wait until halt = '1';
wait for 20 ns;

assert dbg_r1 = x"C001" report "R1 must be C001 after SRA + INCS" severity
failure;
assert dbg_r2 = x"F000" report "R2 must be F000 after OR + NOR" severity
failure;
assert dbg_r3 = x"C001" report "R3 must receive popped stack value" severity
failure;
assert dbg_r4 = x"0000" report "R4 must be zero after NOR FFFF,FFFF" severity
failure;
assert dbg_r5 = x"0000" report "R5 must remain zero because JZ skips 0018h"
severity failure;
assert dbg_r6 = x"0000" report "R6 must remain zero because JMP skips 001Ch"
severity failure;
assert dbg_r7 = x"1111" report "R7 must read first DMA word from
RAM[000Ah]" severity failure;
assert dbg_flags = "1000" report "Final flags must be Z=1, S=0, C=0, O=0"
severity failure;

```

```
    assert dbg_sp = to_unsigned(7, 3) report "Stack pointer must return to empty value
7" severity failure;
    assert dma_done = '1' report "DMA must complete three-word transfer" severity
failure;
    assert bp_hist = "01" report "GHR must become 01 after one taken JZ" severity
failure;

    assert false report "tb_system: TEST PASSED" severity note;
    wait;
end process;
end architecture;
```